

MODUL 9 (HASHING)

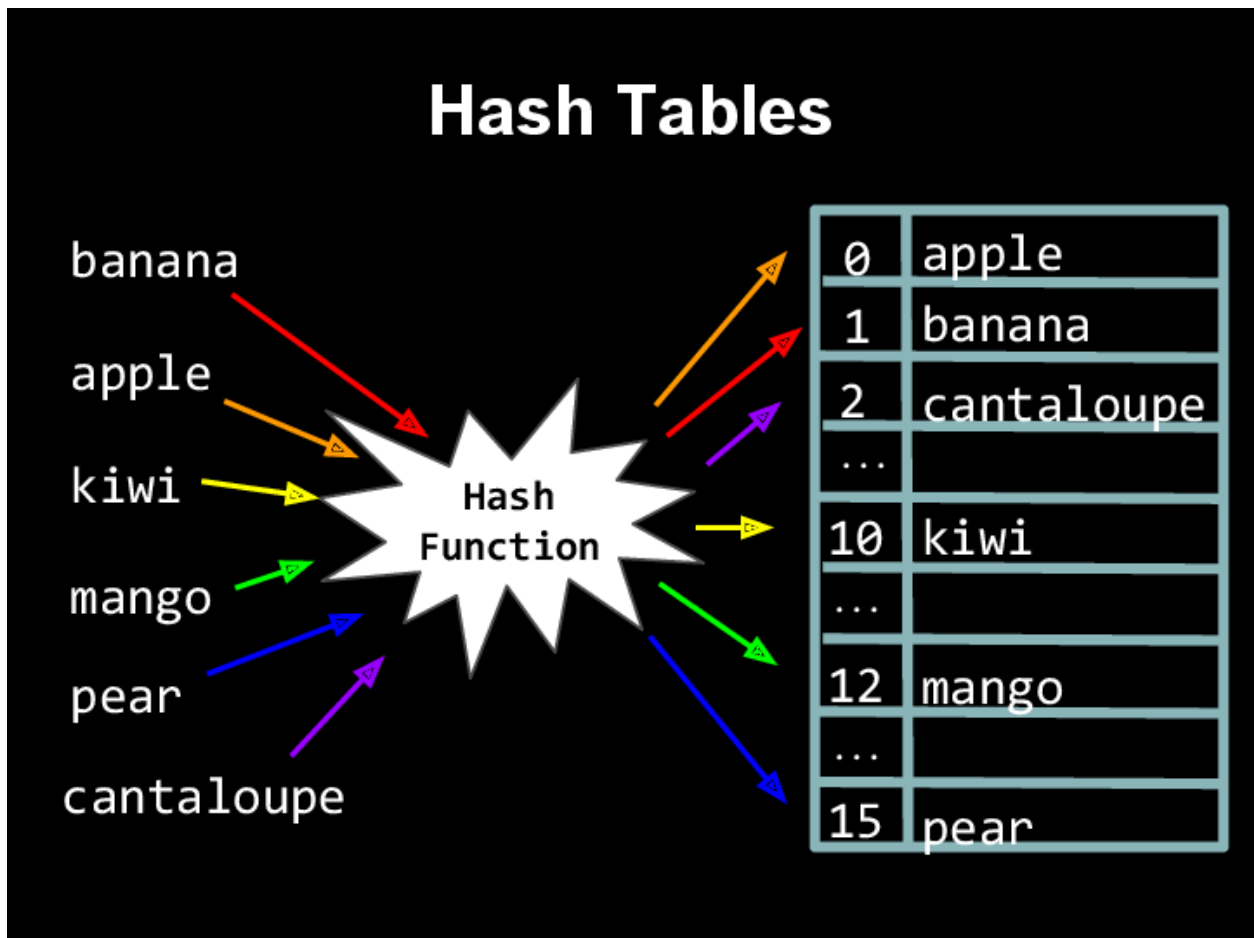
DESKRIPSI TEMA

Merupakan pengembangan dari direct access table. Idennya adalah untuk menggunakan hash function untuk mengkonvert key data seperti misalnya (nim, nomor telepon, dll) menjadi angka dan nomor yang lebih kecil dan menggunakan nomornya sebagai index pada table yang dinamakan Hash table.

Hash table: Array yang menyimpan pointer ke sebuah record nantinya data dicari dengan fungsi hash lalu, didapat datanya.

Hash function: fungsi yang mengconvert angka yang besar menjadi angka kecil.

Gambaran:



Penjelasan: jika dari kata apple huruf depannya adalah a maka ditaruh di hash table nomor 0 dan seterusnya seperti gambar diatas.

Bayangkan seperti gambar pada intro diatas. Jika ditambahkan data bernama anggur, apakah akan terjadi tabrakan? Ya, sehingga data apple akan ter-*replace* dengan data anggur. Data yang ter-*replace* dinamakan dengan collision. Untuk mencegah data tersebut tidak terjadi tabrakan, terdapat beberapa cara yang dapat dilakukan seperti Linear Probing, Rehashing, Quadratic Probing, dan Separate Chaining!

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep hashing dan hash table (C3).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Collision: Linear Probing

Metode untuk menghindari *collision* dengan menggunakan slot yang masih tersedia setelah melakukan hash function, salinlah kodingan berikut ini untuk memahami *linear probing*.

```
#include <stdio.h>
#include <stdbool.h>

int main(int argc, const char** argv){
    int HashT[10] = {0,0,2,0,4,0,0,0,0,0}; //Hash Table
    int inputKey;
    scanf("%d",&inputKey);
    //hash function
    int hash;
    bool check = false;
    hash = inputKey % 10;
    int temp = hash;
    while(1){
        if(HashT[temp]==0){
            HashT[temp] = inputKey;
            break;0
        }
        temp++;
        if(hash==temp){
            printf("HashTable Penuh");
            break;
        }
        if(temp>9){
            temp = 0;
        }
    }
    int i;
    for(i=0;i<10;i++){
        printf("[%d]",HashT[i]);
    }
    return 0;
}
```

Tutorial 2 – Collision: Rehashing

Rehashing adalah suatu cara menghindari *collision* dengan membuat *hash table* baru yang memiliki *size* yang lebih besar. Salinlah potongan kode dibawah ini!



```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <malloc.h>
#include <windows.h>

void sieve(int prime[], int n, int *x){
    int i, j, p;
    int temp[n*2];

    for(i=0; i<n*2; i++){
        temp[i] = 1;
    }

    for(p=2; p*p<=n*2; p++) {
        if (temp[p] == 1) {
            for (i=p*2; i<=n*2; i += p)
                temp[i] = 0;
        }
    }

    j=0;
    for(i=2; i<n; i++){
        if (temp[i] == 1){
            prime[j] = i;
            j++;
        }
    }
    *x = j;
}

int hashFunction(int number, int size){
    return number % size;
}
```

```
int main(int argc, char const *argv[]){
    int *hashTable, *primeTable;
    int i, j, choose, index;
    int primeCount = 0;
    int capacity = 0, percentage = 0;
    int initSize = 10;
    int size = initSize;

    hashTable = (int*) malloc(sizeof(int)*initSize);
    primeTable = (int*) malloc(sizeof(int)*10000);

    for(i=0; i<initSize; i++){
        hashTable[i] = -1;
    }

    for(i=0; i<10000; i++){
        primeTable[i] = 0;
    }

    //generating prime number between 2 and 10000
    sieve(primeTable, 10000, &primeCount);
```

```
index = 0;
do{
    system("cls");
    printf ("1. Input number\n2. Show hash table\n3. Exit\n0. Choose: "); scanf ("%d", &choose);
    if (choose == 1){
        system("cls");
        int number, hash;
        int done = 0;
        printf ("Input one random number : "); scanf ("%d", &number);
        capacity++;
        //using linear probing
        hash = hashFunction(number, size);
        while(!done){
            if (hashTable[hash] == -1){
                hashTable[hash] = number;
                done = 1;
            }
            else hash++;

            if(hash == size) hash = 0;
        }
        percentage = (capacity*100)/size;
        printf("Capacity of hash table = %d%%\n", percentage);

        if (percentage >= 70){
            printf ("hashTable almost full. Making new hash table!\n");

            //moving all data form hashTable1 to temporary array
            int *arrTemp;
            arrTemp = (int*) malloc(sizeof(int)*capacity);
```

```
j = 0;
for(i=0; i<size; i++){
    if (hashTable[i] != -1){
        arrTemp[j] = hashTable[i];
        j++;
    }
}
//free hashTable
free(hashTable);

//new size is twice as many as before
//with the nearest prime number after the size
size *= 2;
//looking for prime
for(i=0; i<primeCount; i++){
    if (primeTable[i] > size){
        size = primeTable[i];
        break;
    }
}

//create new hash table with new size
hashTable = (int*) malloc(sizeof(int)*size);
for(i=0; i<size; i++){
    hashTable[i] = -1;
}
```

```

        //hash all data in arrTemp to hashTable
        for(i=0; i<capacity; i++){
            hash = hashFunction(arrTemp[i], size);
            done = 0;
            while(!done){
                if (hashTable[hash] == -1){
                    hashTable[hash] = arrTemp[i];
                    done = 1;
                }
                else hash++;

                if(hash == size) hash = 0;
            }
        }

        //free unused pointer
        free(arrTemp);
    }
    else{
        //still using hashTable
        printf ("hashTable still can be use\n");
    }
    getch();
}
else if (choose == 2){
    system("cls");
    printf("Content of hash table:\n");
    for(i=0; i<size; i++){
        printf ("hashTable[%d] = %d\n", i, hashTable[i]);
    }
    getch();
}
else if (choose == 0){
    free(hashTable);
    break;
}
else continue;
}while(1);

return 0;
}

```

Note: gunakan garis tepi yang ada di gambar sebagai rujukan identasi!

Jawablah pertanyaan dibawah ini. Simpan file dengan nama **NIM_rehashing.txt**!

1. Mengapa dilakukan *rehashing*?
2. Kapan *rehashing* dilakukan berdasarkan gambar diatas?
3. Sebutkan langkah-langkah melakukan *rehashing* secara singkat, padat, dan jelas!
4. Jelaskan secara singkat apa itu Sieve of Erathosthenes! (1 kalimat)
5. Apakah size yang baru harus mengikuti bilangan prima?

Tutorial 3 – Collision: Quadratic Probing

Metode ini menggunakan sebuah fungsi kuadratik untuk menentukan letak sebuah data yang akan ditaruh di dalam *hash table*. Salinlah potongan kode dibawah ini agar dapat memahami *quadratic probing*.

```
#include <stdio.h>
#include <conio.h>
#include <windows.h>

int quadraticFunction(int number, int n){
    int i, temp;
    temp = n;
    for(i=0; i<n-1; i++){
        temp *= n;
    }
    return (number + temp) % 10;
}

int hashFunction(int number){
    return number % 10;
}
```

```
int main(int argc, char const *argv[]){
    int arr[] = {3, 5, 13, 24, 33, 45, 54, 78, 81, 99, 100};
    int hashTable[10];
    int i, n;
    int hashResult, quadraticResult;

    for(i=0; i<10; i++){
        hashTable[i] = -1;
    }

    for(i=0; i<sizeof(arr)/4; i++){
        hashResult = hashFunction(arr[i]);
        n = 0;
        do{
            quadraticResult = quadraticFunction(hashResult, n);
            if (hashTable[quadraticResult] == -1){
                hashTable[quadraticResult] = arr[i];
                break;
            }
            else if (n == sizeof(arr)/4) break;
            else n++;
        }while(1);
    }

    for(i=0; i<10; i++){
        printf ("hashTable[%d] = %d\n", i, hashTable[i]);
    }

    return 0;
}
```

Penjelasan:

- hashFunction adalah fungsi yang digunakan untuk melakukan hashing
- quadraticFuntion adalah fungsi yang ditambahkan untuk melakukan quadratic probing

Langkah-langkah:

1. Sebuah nilai n pertama-tama akan masuk kedalam hashFunction dan mendapat hash m.
2. Setelah mendapat hash m, dilakukan probing di quadraticFunction dengan mengirimkan nilai m dan nilai awal quadratic (sebut saja x, dan dimulai dari 0).
3. quadraticFuntion diulang sampai hingga data masuk ke dalam hashTable.
4. Nilai m dan x dikirim ke dalam quadraticFunction dan mendapatkan hasil h.
5. Akan dilakukan pengecekan, apakah hashTable[h] sudah ada isi atau belum.
 - Jika belum, maka hashTable[h] akan diisi dengan nilai n.
 - Jika sudah, maka nilai x ditambah 1 (x++) dan mengulang kembali ke langkah 4.

Tutorial 4 – Collision: Separate Chaining

Metode untuk menghindari collision dengan menggunakan array of linked list. Jika data sudah ada maka akan dibuat linked list selanjutnya dengan chaining.

Separate Chaining mirip dengan radix, karena radix memakai fungsi hash untuk menentukan bucketnya.

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <windows.h>

struct numList{
    int number;
    struct numList *next;
};

void insertToChain(int key, struct numList **head){
    struct numList *ptr = (*head);
    struct numList *node = (struct numList*) malloc(sizeof(struct numList));
    node->number = key;
    node->next = NULL;
    if(*head==NULL){
        *head = node;
    }
    else{
        while(ptr->next!=NULL){
            ptr = ptr->next;
        }
        ptr->next = node;
    }
}
```



```
int main(int argc,const char** argv){
    struct numList *HashT[10];
    int i;
    for(i = 0 ; i < 10; i ++){
        HashT[i]=NULL;
    }
    while(1){
        system("cls");
        for(i = 0 ; i <10;i++){
            struct numList *ptr = HashT[i];
            printf("[%d]->",i);
            while(ptr!=NULL){
                printf("(%d)",ptr->number);
                ptr=ptr->next;
            }
            printf("\n");
        }
        int inputKey;
        scanf("%d",&inputKey);
        insertToChain(inputKey,&HashT[inputKey%10]);
    }

    return 0;
}
```

Challenge

Buatlah program yang dapat menyimpan daftar nama, dimana tiap nama disimpan dalam hash table. Entry hash table menggunakan linked list untuk mencegah collision. Hash function yang digunakan akan menggunakan **3 huruf** pertama dari nama untuk menentukan lokasi di hash table (a=0,b=1 dst). Terdapat juga file pendukung dengan nama **datamhs.txt**

Petunjuk : Karena menggunakan 3 huruf pertama dari nama, maka diperlukan array of struct 3 dimensi!

- Main Menu :

```
=====
                        Daftar Mahasiswa
=====
(1). Cari Mahasiswa (berdasar 3 huruf pertama)
(2). Delete Mahasiswa
(3). Tambah Mahasiswa
(0). Exit
Pilih :
```

- Menu 1

```
=====
                          Cari Mahasiswa
=====
Masukkan 3 huruf inisial nama yang ingin dicari:
Leo
Mahasiswa #1
Nama : Leondy
NIM : 16110110079
-----
Mahasiswa #2
Nama : Leonardo
NIM : 16110110038
-----
Mahasiswa #3
Nama : Leonardo Pratama
NIM : 16110110039
-----
Mahasiswa #4
Nama : Leonardus Calvin Hartojo
NIM : 16110110042
-----
Mahasiswa #5
Nama : Leon Christopher
NIM : 16110110094
-----
Tekan tombol apapun untuk melanjutkan...
```

Input harus 3 karakter dan yang ditampilkan adalah data – data nama mahasiswa yang memiliki huruf 3 pertama yang sama dengan input.

Bila input yang dimasukkan tidak merujuk ke data apapun.

```
=====
                          Cari Mahasiswa
=====
Masukkan 3 huruf inisial nama yang ingin dicari:
hah
Anak berinisial hah tidak ditemukan!
Tekan tombol apapun untuk melanjutkan...
```

- Menu 2

```
=====
                          Hapus Mahasiswa
=====
Masukkan nama mahasiswa : Justin Susanto
Mahasiswa dengan nama Justin Susanto berhasil dihapus!
Tekan tombol apapun untuk melanjutkan...
```

Sehingga, jika kembali menu 1 dan mencari inisial nama dari data yang telah dihapus maka akan hilang. Jangan lupa untuk mengatur dimana kondisi jika data yang ingin dihapus tidak ditemukan dan kembali ke menu utama.

```
=====
                        Cari Mahasiswa
=====
Masukkan 3 huruf inisial nama yang ingin dicari:
Jus
Anak berinisial Jus tidak ditemukan!
Tekan tombol apapun untuk melanjutkan...
```

Input nama yang dimasukkan selain menghapus data dalam linked list juga meng-update data yang ada dalam file txt!

- Menu 3

```
=====
                        Tambah Mahasiswa
=====
Masukkan nama mahasiswa baru : Alvin Kecil
Masukkan NIM mahasiswa baru : 12345678910
Mahasiswa baru telah dimasukkan!
Tekan tombol apapun untuk melanjutkan...
```

Mirip seperti menu sebelumnya, jika kembali ke menu 1 dan melakukan pencarian data sesuai inisial nama yang telah ditambahkan maka data yang baru dimasukkan akan muncul.

```
=====
                        Cari Mahasiswa
=====
Masukkan 3 huruf inisial nama yang ingin dicari:
Aly
Mahasiswa #1
Nama : Alvin Janitra
NIM : 16110110077
-----
Mahasiswa #2
Nama : Alvin Yahya
NIM : 16110110107
-----
Mahasiswa #3
Nama : Alvin Kecil
NIM : 12345678910
-----
Tekan tombol apapun untuk melanjutkan...
```

Jangan lupa untuk mengupdate isi didalam txt agar data yang baru ditambahkan masuk.

Format Penamaan: **[NIM]_[Nama]_Challenge.c**

REFERENSI

1. Sieve of Erathosthenes

<https://www.geeksforgeeks.org/sieve-of-eratosthenes/>

2. Open Addressing

<https://www.geeksforgeeks.org/hashing-set-3-open-addressing/>

3. Separate Chaining

<https://www.geeksforgeeks.org/hashing-set-2-separate-chaining/>